

Input files in physical design

by karthik

->Types of input files :

a) Technology input files

- .lib file
- .lef file
- .tlu + file
- .Technology File (.tf)

b) Design input files

- Netlist (.v)
- SDC file
- MMC file
- UPF file
- DEF file

→ Liberty File (.lib)

A **.lib file (Liberty file)** contains the **timing, power, and functional models** of **standard cells**.

It tells tools:

- How fast the cell is
- How much power it consumes
- Its logic function
- Its electrical limits

1. Library-level information

Defined at the top of the .lib file.

Contains:

Library name

Technology parameters

Units

```
library ("saed32hvt_ff0p85v125c") {  
    define(driver_model,library,string);  
    define(def_sim_opt,library,string);  
    define(simulator,library,string);  
    define(always_on,cell,string);  
    define(is_decap_cell,cell,string);  
    define(clk_width,timing,string);  
    technology("cmos");  
    delay_model : "table_lookup";  
    date : "[2010 APRIL 28]";  
    revision : "1.0000000";  
    time_unit : "1ns";  
    leakage_power_unit : "1pW";  
    voltage_unit : "1V";  
    pulling_resistance_unit : "1kohm";  
    current_unit : "1uA";  
    capacitive_load_unit(1.000000, \  
"ff");  
    default_leakage_power_density : 0.000000;  
    input_threshold_pct_rise : 50.000000;  
    output_threshold_pct_rise : 50.000000;  
    input_threshold_pct_fall : 50.000000;  
    output_threshold_pct_fall : 50.000000;  
    slew_lower_threshold_pct_rise : 20.000000;  
    slew_upper_threshold_pct_rise : 80.000000;  
    slew_lower_threshold_pct_fall : 20.000000;  
    slew_upper_threshold_pct_fall : 80.000000;  
    slew_derate_from_library : 1.000000;  
    default_inout_pin_cap : 0.000000;  
    default_cell_leakage_power : 0.000000;  
    default_input_pin_cap : 0.000000;  
}
```

IIAI --

2. Cell information

Each standard cell has its own block.

Contains:

Cell name

Cell area

Cell leakage power

3. Pin information

Defines **pins and their behavior**.

Pin properties:

Direction (input/output/inout)

Capacitance

Max transition

Max capacitance

```
cell (ANTENNA_HVT) {  
  dont_use : true;  
  dont_touch : true;  
  cell_footprint : "ANTENNA ";  
  area : 0.762432 ;  
  cell_leakage_power : 9.185498e+05;  
  driver_waveform_rise : "preDrv";  
  driver_waveform_fall : "preDrv";  
  leakage_power () {  
    when : "INP";  
    value : "9.1854980e+05";  
  }  
  pg_pin (VDD) {  
    voltage_name : "VDD";  
    pg_type : "primary_power";  
  }  
  pg_pin (VSS) {  
    voltage_name : "VSS";  
    pg_type : "primary_ground";  
  }  
  pin (INP) {  
    related_power_pin : "VDD";  
    related_ground_pin : "VSS";  
    direction : "input";  
    fanout_load : 0.131000;  
    rise_capacitance : 0.185743;  
    rise_capacitance_range(0.183849,0.191272);  
    capacitance : 0.185743;  
    fall_capacitance : 0.185743;  
    fall_capacitance_range(0.183849,0.191272);  
  }  
}
```

4. Logic functionality

Defines **what logic the cell performs**.

Example:

```
pin (Y) {  
    direction : output;  
    function : "!(A & B);"  
}
```

Used by:

Logic synthesis

Formal verification

ECO tools

```
max_transition : 1.024000;  
internal_power () {  
    when : "A2&A3";  
    rise_power ("power_inputs_1") {  
        index_1("0.0160000, 0.0320000, 0.0640000, 0.1280000, 0.2560000, 0.5120000, 1.0240000");  
        values("-0.0852276, -1.0364772, -1.9902319, -2.5338872, -2.6724532, -2.0008413, 0.2911270");  
    }  
    fall_power ("power_inputs_1") {  
        index_1("0.0160000, 0.0320000, 0.0640000, 0.1280000, 0.2560000, 0.5120000, 1.0240000");  
        values("0.1302534, -0.9027677, -1.9352835, -2.5876301, -2.8587967, -2.3676032, -0.3516896");  
    }  
}
```

5. Timing information (MOST IMPORTANT)

This is the **core reason PD/STA tools need .lib.**

Timing models include:

Combinational timing

Sequential timing

Clock timing

Timing constraints

Stored as **2D lookup tables** indexed by:

Input slew

Output load

6. Sequential elements (for flops/latches)

Includes:

Setup time

Hold time

Clock-to-Q delay

Recovery/removal

Example:

timing_type : setup_rising;

timing_type : hold_rising;

```
]
latch (IQ2,IQ2N) {
    enable : "SAVE";
    data_in : "Q";
}
pin(Q2) {
    ..
    .
    .
    -
```

```

timing () {
    related_pin : "A1";
    timing_sense : "positive_unate";
    cell_rise ("del_1_7_7") {
        index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");
        index_2("0.1, 0.25, 0.5, 1, 2, 4, 8");
        values("0.0447540, 0.0458206, 0.0476213, 0.0507674, 0.0563600, 0.0660491, 0.0827965", \
            "0.0486896, 0.0497366, 0.0514150, 0.0545056, 0.0600567, 0.0697590, 0.0864749", \
            "0.0565367, 0.0576618, 0.0594047, 0.0625896, 0.0681135, 0.0776786, 0.0943092", \
            "0.0659180, 0.0670654, 0.0689024, 0.0723062, 0.0783407, 0.0885577, 0.1059437", \
            "0.0767123, 0.0781443, 0.0801428, 0.0838908, 0.0907135, 0.1024584, 0.1221303", \
            "0.0894809, 0.0908590, 0.0934383, 0.0978689, 0.1056948, 0.1194869, 0.1428113", \
            "0.1047909, 0.1063051, 0.1086775, 0.1135150, 0.1226201, 0.1391217, 0.1684286");
    }
    rise_transition ("del_1_7_7") {
        index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");
        index_2("0.1, 0.25, 0.5, 1, 2, 4, 8");
        values("0.0229668, 0.0236516, 0.0248756, 0.0273275, 0.0322660, 0.0422780, 0.0627813", \
            "0.0232154, 0.0238878, 0.0251221, 0.0274646, 0.0324437, 0.0425279, 0.0630502", \
            "0.0276783, 0.0282383, 0.0292491, 0.0313642, 0.0358026, 0.0449268, 0.0644383", \
            "0.0373681, 0.0379137, 0.0391123, 0.0408461, 0.0446122, 0.0530311, 0.0705196", \
            "0.0553006, 0.0558640, 0.0568275, 0.0583202, 0.0618035, 0.0696803, 0.0859872", \
            "0.0878531, 0.0883043, 0.0891462, 0.0908113, 0.0944853, 0.1014841, 0.1180243", \
            "0.1481807, 0.1486765, 0.1497141, 0.1514428, 0.1552704, 0.1633110, 0.1812871");
    }
    cell_fall ("del_1_7_7") {
        index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");
    }
}

```


7.a Static Power (Leakage Power)

Definition

Static power is the power consumed **even when the circuit is not switching**.

👉 Power is drawn as long as **VDD is ON**

```
,  
  
cell (A021X2_HVT) {  
    cell_footprint : "A021";  
    area : 2.795584 ;  
    cell_leakage_power : 2.642339e+06;  
    driver_waveform_rise : "preDrv";  
    driver_waveform_fall : "preDrv";  
    leakage_power () {  
        when : "!A1&!A2&!A3";  
        value : "2.3664854e+06";  
    }  
    leakage_power () {  
        when : "!A1&!A2&A3";  
        value : "1.8017191e+06";  
    }  
    leakage_power () {  
        when : "!A1&A2&!A3";  
        value : "2.6423393e+06";  
    }  
    leakage_power () {  
        when : "!A1&A2&A3";  
        value : "1.7905364e+06";  
    }  
    leakage_power () {  
        when : "A1&!A2&!A3";  
        value : "2.4882138e+06";  
    }  
    leakage_power () {  
        when : "A1&!A2&A3";  
        value : "1.7904365e+06";  
    }  
    leakage_power () {
```

7.b Dynamic Power

Definition

Dynamic power is the power consumed **when signals toggle** (0 ↔ 1).

a) Switching power

$$P_{\text{switching}} = \alpha \cdot C_{\text{load}} \cdot V^2 \cdot f$$

- Charging/ discharging output load capacitance

b) Internal power

- Short-circuit current
- Internal node capacitance

```
,  
  
cell (A021X2_HVT) {  
    cell_footprint : "A021";  
    area : 2.795584 ;  
    cell_leakage_power : 2.642339e+06;  
    driver_waveform_rise : "preDrv";  
    driver_waveform_fall : "preDrv";  
    leakage_power () {  
        when : "!A1&!A2&!A3";  
        value : "2.3664854e+06";  
    }  
    leakage_power () {  
        when : "!A1&!A2&A3";  
        value : "1.8017191e+06";  
    }  
    leakage_power () {  
        when : "!A1&A2&!A3";  
        value : "2.6423393e+06";  
    }  
    leakage_power () {  
        when : "!A1&A2&A3";  
        value : "1.7905364e+06";  
    }  
    leakage_power () {  
        when : "A1&!A2&!A3";  
        value : "2.4882138e+06";  
    }  
    leakage_power () {  
        when : "A1&!A2&A3";  
        value : "1.7904365e+06";  
    }  
    leakage_power () {
```

8. Wire load model

- **Wire Load** is a **pre-layout interconnect model** used mainly during **logical synthesis**.
- Since **actual routing is not yet done**, wire load provides an **estimate of interconnect delay**.

What it contains?

- Estimated Wire Length
- Resistance per Unit Length
- Capacitance per Unit Length
- Fanout-Based Models

Wire load models are grouped by:

- Low fanout
- Medium fanout
- High fanout nets

Each group has different RC values.

```
wire_load ("16000") {  
    capacitance : 0.000512;  
    resistance : 1.153000e-03;  
    area : 0.010000;  
    slope : 108.437647;  
    fanout_length("1", \  
        "15.1043810");  
    fanout_length("2", \  
        "34.6068000");  
    fanout_length("3", \  
        "56.4162210");  
    fanout_length("4", \  
        "80.8425080");  
    fanout_length("5", \  
        "108.1955250");  
    fanout_length("6", \  
        "138.7851360");  
    fanout_length("7", \  
        "172.9212050");  
    fanout_length("8", \  
        "210.9135960");  
    fanout_length("9", \  
        "253.0721730");  
    fanout_length("10", \  
        "299.7068000");  
    fanout_length("11", \  
        "351.1273410");  
    fanout_length("12", \  
        "407.6436600");  
    fanout_length("13", \  
        "469.5656210");  
    fanout_length("14", \  
        "537.2030880");  
}
```

→ **Library Exchange Format file (.lef) (cell lef + tech lef)**

A .lef (Library Exchange Format) file contains the physical information of standard cells and macros.

→ **CELL LEF:**

1. Cell physical dimensions

- Cell width and height
- Used for placement and floorplanning

```
VERSION 5.4 ;  
NAMESCASESENSITIVE ON ;  
BUSBITCHARS "⌚" ;  
DIVIDERCHAR "/" ;
```

```
MACRO TNBUFFX2_HVT  
  CLASS CORE ;  
  SOURCE USER ;  
  ORIGIN 0 0 ;  
  SIZE 2.128 BY 1.672 ;  
  SYMMETRY X Y R90 ;  
  SITE unit ;
```

```
PIN EN  
  DIRECTION INPUT ;  
  USE SIGNAL ;  
  PORT  
    LAYER M1 ;  
    RECT 0.3240 1.3730 0.7360 1.4230 ;  
    RECT 0.5530 1.4230 0.6630 1.5750 ;  
  END  
  PORT  
    LAYER CO ;  
    RECT 0.6630 1.3770 0.7050 1.4190 ;
```

2. Pin information (physical)

- Pin location
- Pin shape
- Pin metal layer

Ex:

```
PIN A  
  DIRECTION INPUT ;  
  PORT  
    LAYER M1 ;  
  END  
END A
```

3. bstructions (OBS)

Areas where routing is **not allowed**

Prevent metal overlap on cells

4. Site information

Defines **placement grid**

Controls row-based placement

```
END CLK
```

```
OBS
```

```
LAYER C0 ;
```

```
RECT 5.0710 0.6080 5.1130 0.6500 ;
```

```
RECT 1.4990 1.0920 1.5410 1.1340 ;
```

```
RECT 2.4870 0.1000 2.5290 0.1420 ;
```

```
RECT 0.7390 0.5160 0.7810 0.5580 ;
```

```
RECT 0.2830 0.3670 0.3250 0.4090 ;
```

```
RECT 2.0310 1.5300 2.0730 1.5720 ;
```

```
RECT 1.1950 0.5160 1.2370 0.5580 ;
```

```
RECT 1.3470 0.9920 1.3890 1.0340 ;
```

```
RECT 5.0710 0.6080 5.1130 0.6500 ;
```

```
RECT 4.9190 0.6080 4.9610 0.6500 ;
```

```
RECT 2.6390 0.1000 2.6810 0.1420 ;
```

```
RECT 3.4750 1.1640 3.5170 1.2060 ;
```

```
RECT 4.2350 0.9820 4.2770 1.0240 ;
```

```
RECT 0.8910 0.3320 0.9330 0.3740 ;
```

```
RECT 3.2470 1.5220 3.2890 1.5640 ;
```

```
RECT 2.4110 1.2520 2.4530 1.2940 ;
```

```
RECT 0.2830 0.3670 0.3250 0.4090 ;
```

```
RECT 2.2590 0.5060 2.3010 0.5480 ;
```

```
RECT 2.2590 0.4140 2.3010 0.4560 ;
```

5. Macro blocks

- Large blocks (SRAM, IPs)
- Size, pins, blockages

→ Technology LEF (tech.lef)

Contains:

- Metal layers
- Via definitions
- Routing rules
- Tracks

1 Metal Layers (M1, M2, M3...)

- Direction (horizontal / vertical)
- Width, spacing, pitch
- Resistance & capacitance (for routing)

2 Via Definitions

- Connections between metal layers (e.g., M1–M2)
- Via size and enclosure rules

3 Tracks

- Routing grid used by the router
- X/Y direction tracks for each metal layer

4 Top Metal Information

- Which metal layer is the highest routing layer

→ TLU + file:

A TLU+ (Table Look-Up Plus) file contains interconnect RC (Resistance & Capacitance), cross talk data used for accurate timing and power analysis

1. Wire Resistance Tables

Resistance per unit length

Depends on:

- Metal layer

- Wire width

- Wire spacing

2. Wire Capacitance Tables

Includes:

- Ground capacitance

- Fringe capacitance

- Coupling capacitance between adjacent wires

3. Via Resistance & Capacitance

Single-cut via RC

Multi-cut via RC

4. Layer-to-Layer Coupling Data

Capacitance between:

M1 $\leftrightarrow$ M2

M2 $\leftrightarrow$ M3

Accounts for vertical coupling

5. Process Corners

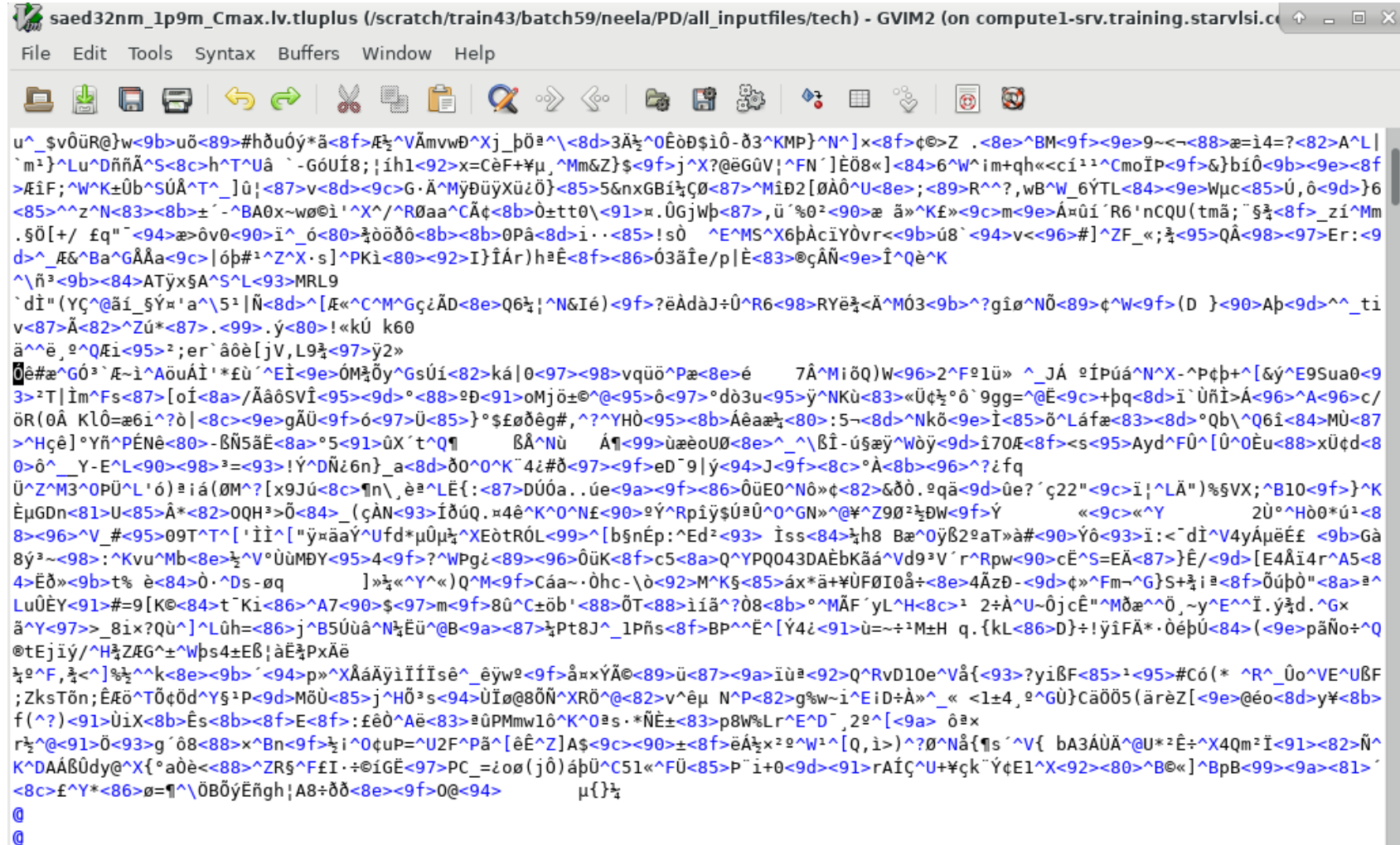
Max TLU+ $\rightarrow$ worst-case RC (setup analysis)

Min TLU+ $\rightarrow$ best-case RC (hold analysis)

6. Environment-Dependent Data

RC variation based on:

- Temperature
- Process variation



```
saed32nm_1p9m_Cmax.lv.tluplus (/scratch/train43/batch59/neela/PD/all_inputfiles/tech) - GVIM2 (on compute1-srv.training.starvlsi.c
File Edit Tools Syntax Buffers Window Help
u^_ $v0üR@}w<9b>uö<89>#hðuóý*ä<8f>Æ½^VÄmvwð^Xj_böæ^\\<8d>3Ä½^0Êðð$î0-ð3^KMP}>^N^]x<8f>ç@>Z .<8e>^BM<9f><9e>9~<-<88>æ=i4=?<82>A^L|
`m¹}>^Lu^DññÄ^S<8c>h^T^Uâ ` -GóUÍ8;¡íhl<92>x=CèF+¥µ_`Mm&Z}>$<9f>j^X?@èGûV¡^FN^']Ê08«]><84>6^W^im+qh«<ci¹¹^CmoİP<9f>&}bí0<9b><9e><8f
>ÆiF;^W^K±Üb^SÚÄ^T^_]û¡<87>v<8d><9c>G·Ä^MÿðüÿXü¿0}><85>5&nxBi½ç0<87>^Mið2[0Ä0^U<8e>;<89>R^^?,wB^W_6YTL<84><9e>Wµc<85>û,ð<9d>}6
<85>^z^N<83><8b>±`-^BA0x~wø@i'^X^/^R0aa^CÄç<8b>0±tt0\<91>¤.ÜGjWp<87>,ü`%0²<90>æ ä»^Kf»<9c>m<9e>Ä¤úí'R6'nCQU(tmâ;`$½<8f>_zíMm
.ş0[+/- fq`~<94>æ>ðv0<90>i^_ó<80>¾ððð0<8b><8b>0Pâ<8d>i..<85>!s0 ^E^MS^X6bÄciY0vr<<9b>ú8`<94>v<<96>#]>]ZF_«;¾<95>QÄ<98><97>Er:<9
d>^_Æ&^Ba^GÄÄa<9c>|óp#¹^Z^X·s]^PKi<80><92>I}İÁr)hæÊ<8f><86>03äİe/p|Ê<83>@çÄÑ<9e>İ^Qè^K
^\\ñ³<9b><84>ATÿx$A^S^L<93>MRL9
`dİ"(Yç@äi_şY¤'a^\\5¹|Ñ<8d>^ [Æ<^C^M^Gç¿ÄD<8e>Q6¾¡^N&İé)<9f>?æÄdàJ+Ü^R6<98>RYè¾¿Ä^M03<9b>^?giø^Nö<89>ç^W<9f>(D }<90>Aþ<9d>^_ti
v<87>Ä<82>^Zú*^<87>.<99>.ý<80>!«kü k60
ä^ë,^QÆi<95>²;er`âðè[jv,L9¾<97>ÿ2»
0è#æ^G0³`Æ~i^AöuÄİ'`fù'^Eİ<9e>0M¾0y^Gsúí<82>ká|0<97><98>vqüö^Pæ<8e>é 7Ä^MiðQ)W<96>2^Fè1ü» ^_JÄ èİPúá^N^X-^Pçþ+^[&ý^E9Sua0<9
3>²T|İm^Fs<87>[oí<8a>/ÄâðSVİ<95><9d>°<88>ºð<91>oMjô±@^@<95>ð<97>°dò3u<95>ÿ^NKù<83>«Üç½°ð`9gg=^@Ê<9c>+þq<8d>i`Üñİ>Á<96>^A<96>c/
öR(0Ä Kl0=æ6i^?ò|<8c><9e>gÄÜ<9f>ó<97>Ü<85>}>°$fððèg#,^?^YH0<95><8b>Äéæ¾¿<80>:5~<8d>^Nkð<9e>İ<85>ð^Láfæ<83><8d>°Qb\^Q6i<84>MÜ<87
>^Hçè]^Yñ^PÉNè<80>-ßÑ5äÈ<8a>°5<91>ûX`t^Q¶ BÄ^Nü Ä¶<99>ûæèou0<8e>^_\\ßİ-úsæÿ^Wòÿ<9d>î70Æ<8f><s<95>Ayd^FÜ^[Ü0Èu<88>xÜçd<8
0>ð^__Y-E^L<90><98>³=<93>!Y^DÑ¿6n}_a<8d>ð0^0^K`4i#ð<97><9f>eD`9|ý<94>J<9f><8c>°Ä<8b><96>^?¿fç
Ü^Z^M3^0PÜ^L'ó)¤iá(0M^?[x9Jú<8c>¶n\,èè^LÊ{:<87>DÜ0a..úe<9a><9f><86>0üE0^Nð»ç<82>&ðð.ºqä<9d>úe?`ç22"<9c>i;^LÄ")%şVX;^B10<9f>}>^K
ÈµGðn<81>U<85>Ä*^<82>0QH³>0<84>_(çÄN<93>İðúQ.¤4è^K^0^Nf<90>ºY^Rpîÿ$Ü¤Ü0^0^GN»^@ç^Z9ð²¾ðW<9f>Ý «<9c>«^Y 2Ü°^Hò0*ú¹<8
8><96>^V_#<95>09T^T^['İİ^["ÿ¤ääY^Ufd*µü¾¿^XEòtR0L<99>^[bşnÉp:^Ed²<93> İss<84>¾h8 Bæ^0ÿB2ªaT>à#<90>Ýò<93>İ:<`dİ^V4yÁµèÉf <9b>Gà
8ÿ³~<98>:^Kvu^Mb<8e>¾^V°ÜùMðY<95>4<9f>?^WPg¿<89><96>0üK<8f>c5<8a>Q^YPQ043DAèbKáá^Vd9³V`r^Rpw<90>cÈ^S=EÄ<87>}>Ê/<9d>[E4Äi4r^A5<8
4>Èð»<9b>t% è<84>ð·^Ds-øq ]»¾«^Y^«)Q^M<9f>Cáa~·0hc-\ò<92>M^K$<85>áx*ä+þÜF0İ0ä><8e>4Äzð-<9d>ç»^Fm~^G}>S+¾i¤<8f>0üþ0"<8a>¤^
LuÜEY<91>#>9[K@<84>t`Ki<86>^A7<90>$<97>m<9f>8ûC±öb'<88>0T<88>iİä^?08<8b>°^MÄF`yL^H<8c>¹ 2÷Ä^U~0jçÈ^Mðæ^H0,~y^E^Äİ.ý¾d.^Gx
ä^Y<97>>_8ix?Qù^]^Lùh=<86>j^B5Üúä^N¾Èü^@B<9a><87>¾Pt8J_1Pñs<8f>BP^^È^[Y4¿<91>ù=÷+¹M±H q.{kL<86>D}>÷!ÿİFÄ*·0épÜ<84>(<9e>päÑo÷^Q
@tEjİÿ/^H¾ZÆG±^Wps4±EB¡àÈ¾PxÄè
¾º^F,¾<^]>¾¾^k<8e><9b>`<94>p»^XÄäÄÿiİİİè^_èÿwº<9f>â¤xYÄ@<89>ü<87><9a>İü¤<92>Q^Rvd10e^Vâ{<93>?yißF<85>¹<95>#Có(* ^R^_Üo^VE^UßF
;ZksTön;ÈÄö^T0ç0d^Yş¹P<9d>MöÜ<85>j^H0³s<94>Üİð080Ñ^XR0^@<82>v^èµ N^P<82>gşw~i^Eİd÷Ä»^_« <1±4,º^GÜ)Cä005(ärèZ<9e>@éo<8d>y#<8b>
f(^?)<91>ÜiX<8b>Ès<8b><8f>E<8f>:fè0^Aè<83>èUPMmwł0^K^0¤s·^ÑÈ±<83>p8W%Lr^E^D`²º^[<9a> 0¤x
r¾¿<91>0<93>g`08<88>x^Bn<9f>¾i^0çuP=^U2F^Pä^[èÈ^Z]A$<9c><90>±<8f>èÄ½x²º^W¹^[Q,i>)^?0^Nä{¶s`^V{ bA3AÜÄ^@U*²È÷^X4Qm²İ<91><82>Ñ^
K^DAÁß0dy@^X{^ºa0è<<88>^ZRş^FİI÷÷igE<97>PC=¿oø(j0)ápÜ^C51«^FÜ<85>P`i+0<9d><91>rAİç^U+¥çk`ÝçE1^X<92><80>^B@«]^BpB<99><9a><81>`
<8c>£^Y*^<86>ø=¶\\0B0ÿÈñgh¡A8÷ðð<8e><9f>0@<94> µ{}}¾
@
@
```

→ Technology file (.tf)

A **.tf (Technology File)** contains **manufacturing rules and physical layer definitions** required for **routing and layout**

1. Layer Definitions

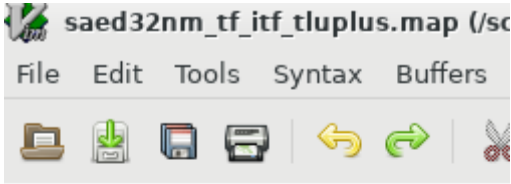
- Metal layers (M1, M2, M3 ...)
- Via layers
- Cut layers
- Diffusion and poly layers

Example info:

Layer name

Layer type (routing / cut)

Preferred routing direction (H / V)



```
saed32nm_tf_itf_tluplus.map (/sc
File Edit Tools Syntax Buffers
conducting_layers
    DIFF      DIFF
    P0         POLY
    M1         M1
    M2         M2
    M3         M3
    M4         M4
    M5         M5
    M6         M6
    M7         M7
    M8         M8
    M9         M9
    MRDL      MRDL
via_layers
    C0         POLYCONT
    VIA1       VIA1
    VIA2       VIA2
    VIA3       VIA3
    VIA4       VIA4
    VIA5       VIA5
    VIA6       VIA6
    VIA7       VIA7
    VIA8       VIA8
    VIARDL     VIARDL
marker_layers
    metal1_pin
    metal2_pin
    metal3_pin
    metal4_pin
    metal5_pin
    metal6_pin
```

2. Routing Rules

- Minimum width
- Minimum spacing
- Minimum pitch
- Minimum area rules

👉 Ensures **DRC-clean routing**

3. Via Definitions

- Via size (cut width/height)
- Enclosure rules
- Spacing between vias
- Multi-cut via rules

⇒ Design data inputs

➔ Netlist (.v)

- A **Netlist** is a **gate-level description of the design** showing **how cells are logically connected**.
- It is the **output of synthesis** and the **main input to Physical Design**.

1. Standard Cell Instances

Logic cells like:

AND, OR, NAND, NOR

Flip-flops, latches

Buffers, inverters

Example:

```
U1 AND2X1 ( .A(n1), .B(n2), .Y(n3) );
```

```

        .C1(\mult_x_28/n410 ), .C0(\mult_x_28/n402 ), .S(\mult_x_28/n403 ), ,
FADDX1_LVT \mult_x_28/U327 ( .A(\mult_x_28/n560 ), .B(\mult_x_28/n527 ),
        .CI(\mult_x_28/n538 ), .C0(\mult_x_28/n406 ), .S(\mult_x_28/n407 ) );
FADDX1_LVT \mult_x_28/U326 ( .A(\mult_x_28/n549 ), .B(\mult_x_28/n414 ),
        .CI(\mult_x_28/n409 ), .C0(\mult_x_28/n404 ), .S(\mult_x_28/n405 ) );
HADDX1_LVT \mult_x_28/U328 ( .A0(\mult_x_28/n516 ), .B0(\mult_x_28/n505 ),
        .C1(\mult_x_28/n408 ), .S0(\mult_x_28/n409 ) );
OAI21X1_LVT \mult_x_28/U201 ( .A1(\mult_x_28/n168 ), .A2(\mult_x_28/n170 ),
        .A3(\mult_x_28/n169 ), .Y(\mult_x_28/n167 ) );
OAI21X1_LVT \mult_x_28/U215 ( .A1(\mult_x_28/n179 ), .A2(\mult_x_28/n176 ),
        .A3(\mult_x_28/n177 ), .Y(\mult_x_28/n175 ) );
HADDX1_LVT \mult_x_28/U334 ( .A0(\mult_x_28/n563 ), .B0(\mult_x_28/n552 ),
        .C1(\mult_x_28/n420 ), .S0(\mult_x_28/n421 ) );
FADDX1_LVT \mult_x_28/U329 ( .A(\mult_x_28/n418 ), .B(\mult_x_28/n415 ),
        .CI(\mult_x_28/n416 ), .C0(\mult_x_28/n410 ), .S(\mult_x_28/n411 ) );
FADDX1_LVT \mult_x_28/U332 ( .A(\mult_x_28/n562 ), .B(\mult_x_28/n551 ),
        .CI(\mult_x_28/n420 ), .C0(\mult_x_28/n416 ), .S(\mult_x_28/n417 ) );
HADDX1_LVT \mult_x_28/U331 ( .A0(\mult_x_28/n528 ), .B0(\mult_x_28/n517 ),
        .C1(\mult_x_28/n414 ), .S0(\mult_x_28/n415 ) );
HADDX1_LVT \mult_x_28/U333 ( .A0(\mult_x_28/n540 ), .B0(\mult_x_28/n529 ),
        .C1(\mult_x_28/n418 ), .S0(\mult_x_28/n419 ) );
FADDX1_LVT \mult_x_28/U330 ( .A(\mult_x_28/n561 ), .B(\mult_x_28/n539 ),
        .CI(\mult_x_28/n550 ), .C0(\mult_x_28/n412 ), .S(\mult_x_28/n413 ) );
OAI21X1_LVT \mult_x_28/U16 ( .A1(\mult_x_28/n37 ), .A2(\mult_x_28/n47 ),
        .A3(\mult_x_28/n38 ), .Y(\mult_x_28/n36 ) );
NOR2X0_HVT \mult_x_28/U478 ( .A1(n214), .A2(n196), .Y(N751) );
NOR2X0_LVT \mult_x_28/U476 ( .A1(n214), .A2(n202), .Y(\mult_x_28/n563 ) );
NOR2X0_HVT \mult_x_28/U475 ( .A1(n214), .A2(n200), .Y(\mult_x_28/n562 ) );
NOR2X0_LVT \mult_x_28/U474 ( .A1(n214), .A2(n4365), .Y(\mult_x_28/n561 ) );
NOR2X0_HVT \mult_x_28/U473 ( .A1(n214), .A2(n4363), .Y(\mult_x_28/n560 ) );

```

2. Cell Connections (Nets)

- Shows **which pin is connected to which**
Nets are wires between cells

Example:

wire n1, n2, n3;

3. Module Definitions

Top module

Sub-modules (if any)

Example:

module top (clk, rst, in1, out1);

```
// WIRES AND REGS
wire lreset;
wire hreset;
wire [1:0] sout;
wire [1:0] sout_sync;
wire [15:0] hash_data_out;
wire [15:0] t_hash_data_out;
wire [31:0] t_ack_signals;
wire [31:0] switch_ack_signal;
```

```
module bit_coin (
    hclk,
    lclk,
    reset,
    secure_data_in,
    hash_key,
    data_valid,
    sleep_signals,
    isolation_signals,
    retention_signals,
    test_mode,
    scan_enable,
    sipo_scan_in,
    piso_scan_in,
    hv_scan_in,
    lv_scan_in,
    sipo_scan_out,
    piso_scan_out,
    hv_scan_out,
    lv_scan_out,
    secure_data_out,
    memory_address,
    data_ready,
    power_ack_signals
);
```

4. Port Declarations

- Input ports
- Output ports
- Inout ports

Example:

input clk, rst;

output out1;

5. Power and Ground Connections

- VDD, VSS
- Power pins of cells

Example:

.VDD(VDD), .VSS(VSS)

```
// INPUTS
input hclk;
input lclk;
input reset;
input data_valid;
input [31:0] secure_data_in;
input [31:0] hash_key;
```

```
// DFT INPUTS
input test_mode;
input scan_enable;
input sipo_scan_in;
input piso_scan_in;
inout hv_scan_in;
```

→ SDC file

- **SDC (Synopsys Design Constraints)** file contains **timing and design constraints**.
- It tells the tool **how fast the design must run** and **under what conditions**.

1. Timing Constraints

a) Create Clock (Primary Clock)

- Defines clock coming from a **port**

Example:

```
create_clock -name CLK -period 10 [get_ports clk]
```

Period = 10 ns → 100 MHz

All setup/hold timing depends on this

 **Without clocks → no timing analysis**

b) Create Generated Clock

- Clock derived from another clock (PLL/divider/mux).

Example:

```
create_generated_clock -name CLK_DIV2 -source [get_ports clk] -divide_by 2 [get_pins u_div/clock_out]
```

c) Input Delay

- Models arrival time of **external data**.

Example:

```
set_input_delay 2 -clock CLK [get_ports data_in]
```

d) Output Delay

- Models required time for **output data**.

Example:

```
set_output_delay 3 -clock CLK [get_ports data_out]
```

2. CLOCK EXCEPTIONS

- Clock exceptions define **relationships between clocks**

a) Clock Uncertainty

- the maximum expected deviation in a clock signal's arrival time, combining jitter, skew and safety margins to ensure robust timing.

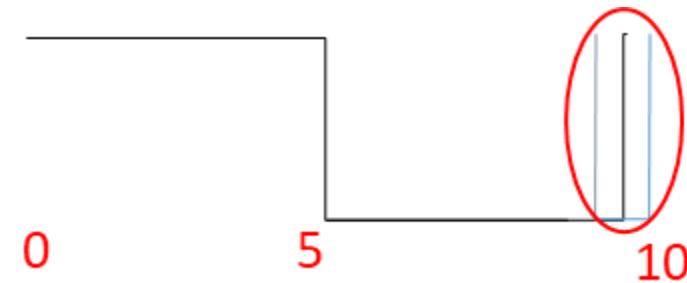
What it models:

- Clock jitter
- Skew
- PVT variation

Example:

```
set_clock_uncertainty 0.2 [get_clocks CLK]
```

📌 Reduces effective timing budget.



b) Clock Latency

- Defines delay of clock reaching FFs.

Source latency (before CTS):

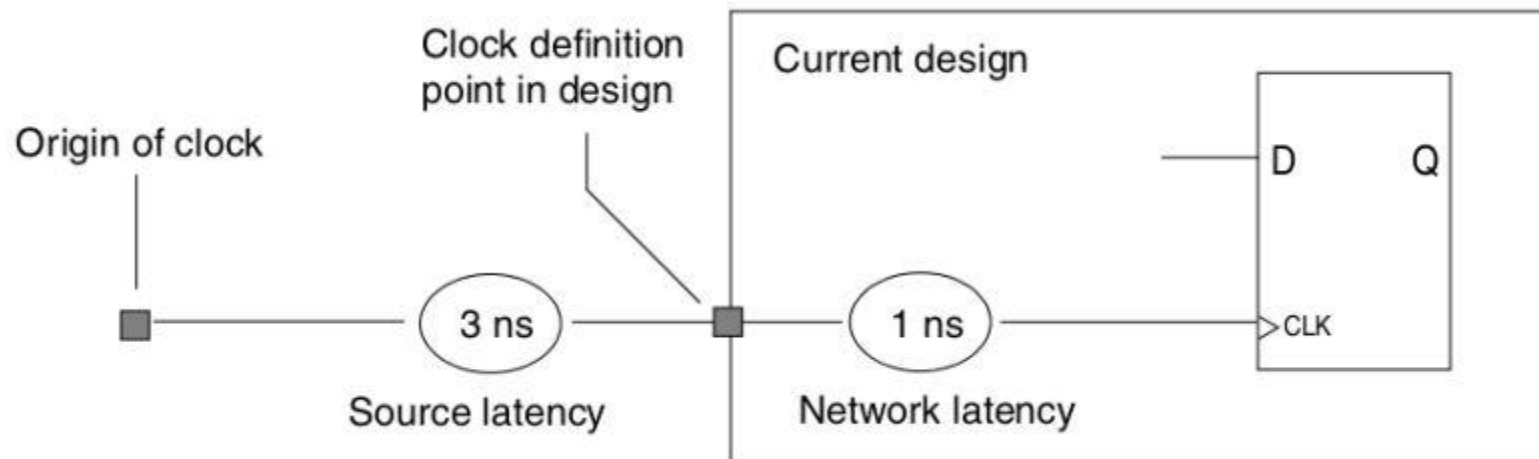
Example:

```
set_clock_latency -source 1.0 [get_clocks CLK]
```

Network latency (after CTS):

Example:

```
set_clock_latency 0.5 [get_clocks CLK]
```



c) Clock groups

Type	Definition	Simple Example
Asynchronous	Clocks that are not related at all . They come from different sources and don't run in sync.	<code>clk_cpu</code> from PLL1 and <code>clk_usb</code> from external crystal
Exclusive	Clocks where only one clock works at a time depending on the mode.	Functional clock active in normal mode, scan clock active in test mode
Physically Exclusive	Clocks where hardware ensures only one reaches the circuit (like a clock MUX).	<code>clk1</code> and <code>clk2</code> connected to a clock MUX; only one selected
Logically Exclusive (Mode-based)	Clocks that are mutually exclusive because of control logic or mode , even if no physical MUX is obvious.	Low-power mode clock vs High-performance mode clock
Synchronous (Related clocks)	Clocks that are related and derived from the same source , so timing between them must be checked .	<code>clk</code> and <code>clk_div2</code> from the same PLL

3. TIMING EXCEPTIONS

- Timing exceptions apply to **data paths**
- Timing exceptions prevent tools from **over-optimizing unreal paths**.

◆ Multicycle Path

Allows data to take **multiple cycles**.

Example:

```
set_multicycle_path 2 -setup -from A -to B  
set_multicycle_path 1 -hold -from A -to B
```

◆ False Path

Path that **never happens** functionally.

Example:

```
set_false_path -from [get_ports test_en] -to [get_registers *]
```



Tool ignores setup & hold checks.

Example:

set_min_delay 1 -from A -to B

```
# CLOCK GROUP
```

MAX DELAY

```
# FALSE PATH
```

```
# MULTICYCLE PATH
```

```
set_multicycle_path -hold -setup 4 -from secure_data_in* -to secure_data_out*
set_multicycle_path -hold -setup 4 -from isolation_signal* -to data_ready
set_multicycle_path -hold -setup -to [get_pins *SI* -hier] 4
set_multicycle_path -hold -setup -from isolation_signals* 20
set_multicycle_path -hold -setup -from bit_secure_*/slice_*/sipo_bit/wr_tmp_reg/CLK -to bit_secure_*/slice_*/sipo_bit/wr_0_re
g/D 4
```

4. DESIGN RULE CONSTRAINTS

- These do **NOT** define timing directly.
- They control **signal quality and physical feasibility**.

❑ Max Fanout

What it controls:

Limits how many loads one driver can drive

```
set_max_fanout 10 [get_designs *]
```

❑ Max Transition (Max Slew)

What it controls:

Maximum rise/fall time of a signal.

```
set_max_transition 0.2 [get_designs *]
```

❑ Max Capacitance (sometimes used)

Controls total load on a net.

```
set_max_capacitance 0.3 [get_ports data_out]
```

→ DEF file

- A DEF file contains the physical layout information of the design.
- It shows where things are placed on silicon.

1. Die Area and Core Area

- Chip boundary
- Core boundary

DIEAREA (0 0) (2000 2000);

2. Rows

Defines **standard cell rows**.

ROW ROW_1 coreSite 0 0 N DO 100 BY 1 STEP 0 1;

```
#
# IC Compiler II write_def
# Release      : K-2015.06
# User Name    : amins
# Date        : Thu Jul  9 09:01:25 2015
#
VERSION 5.7 ;
DIVIDERCHAR "/" ;
BUSBITCHARS "[]" ;
DESIGN ORCA_TOP ;
UNITS DISTANCE MICRONS 1000 ;
DIEAREA ( 0 0 ) ( 1013080 0 ) ( 1013080 780824 ) ( 416480 780824 ) ( 416480 596904 ) ( 0 596904 ) ;
ROW _row_1 unit 10032 9880 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_2 unit 10032 11552 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_3 unit 10032 13224 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_4 unit 10032 14896 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_5 unit 10032 16568 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_6 unit 10032 18240 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_7 unit 10032 19912 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_8 unit 10032 21584 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_9 unit 10032 23256 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_10 unit 10032 24928 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_11 unit 10032 26600 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_12 unit 10032 28272 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_13 unit 10032 29944 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_14 unit 10032 31616 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_15 unit 10032 33288 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_16 unit 10032 34960 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_17 unit 10032 36632 FS D0 6533 BY 1 STEP 152 0 ;
ROW _row_18 unit 10032 38304 N D0 6533 BY 1 STEP 152 0 ;
ROW _row_19 unit 10032 39976 FS D0 6533 BY 1 STEP 152 0 ;
```

3. Component Placement

Cell name

Location (x, y)

Orientation

- U1 AND2X1 + PLACED (100 200) N ;

4. IO Pin Locations

Pin name

Location

Layer

- clk + FIXED (0 500) M2 ;

5. Power and Ground Nets

VDD, VSS routing

Power straps

- VDD (U1 VDD) (U2 VDD);

6. Routed Wires

Routing layers

Wire paths

Via locations

+ ROUTED M2 (100 200) (300 200) VIA12 ;

7. Blockages

Routing blockages

Placement blockages

8 Track Definitions

Routing tracks for metals

Track pitch and direction

TRACKS X 0.0 DO 200 STEP 0.14 LAYER M2 ;

→.mmmc file

An **.mmmc (Multi-Mode Multi-Corner)** file tells EDA tools **how to perform timing analysis for all operating conditions of a chip.**

1. Modes

- Different design modes
- Example: *functional, scan*

2. Process-Voltage-Temperature (PVT) Corners

- Conditions under which the chip operates
- Example: *slow, typical, fast*

3. Timing Libraries (.lib)

- Specific libraries used for each corner

4. RC Corners

- Wire resistance and capacitance data
- Uses TLU+ / QRC files

5. Constraint Files (SDC)

- Clock and timing constraints for each mode

➔ Unified power format file (.upf)

A UPF (Unified Power Format) file describes the **power intent of a design** — how power is managed to save energy.

1. Power Domains

- Separate regions with independent power control
- Example: CORE, MEMORY, IO

2. Power Supplies

- Power and ground nets
- Example: VDD, VSS

3. Power Switches

- Logic used to turn power ON/OFF for domains

4. Isolation Cells

- Prevents unknown (X) values when a domain is OFF

5. Level Shifters

- Safely transfer signals between different voltage domains

6. Retention Cells

- Save state when power is turned OFF

7. Power States

- Defines ON, OFF, SLEEP modes

8. Power Intent Rules

- When and where low-power cells must be inserted